

1976'nın Ekonomisi: Kod İncelemesinin Kökeni ve Sürekliliği

ÖFY

August 2026

Özet

Kod incelemesi (*code review*) 1976'da, yazılımın dağıtım maliyetinin yüksek ve geri alınmasının imkânsız olduğu bir dönemde, hatayı üretime çıkmadan yakalamak için tasarlanmıştır. Bu ekonomik gerekçe sürekli teslimat araçlarıyla birlikte ortadan kalkmış, ancak pratik varlığını sürdürmüştür. Bugün uygulanan biçim, özgün yöntemin ne kendisi ne de hafifletilmiş bir sürümüdür; yalnızca adı ortaktır. Ampirik çalışmalar, modern kod incelemesinin beklenen işlevini yerine getirmediğini, buna karşılık kimsenin planlamadığı bir yan etki ürettiğini göstermektedir: bilgi dağılımı. Kod incelemesinin taşıdığı işlevler ayrıştırıldığında bunların üçü otomasyona ve yazıya devredilebilir; devredilemeyen dördüncüsü, bir yazılım biriminin gerçek riskidir. İncelemenin organizasyonda aldığı üçüncü biçim olan onay ve veto yetkisi, ne hata yakalamaya ne bilgi dağılımına hizmet eder; alan gözlemlerinde “ego çatışması” olarak adlandırılan olgunun kaynağı budur. İnceleme yükümlülüğünün kaldırılabilceği koşullar, değişikliklerin geri alınabilirlik ve etki alanı eksenlerinde sınıflandırılmasıyla belirlenebilir. Geri alnamayan değişiklikler için doğru müdahale inceleme değil, o değişikliği geri alınabilir hâle getiren tasarım kararıdır.

Giriş

Kod incelemesi, bir geliştiricinin yazdığı kaynak kodun, kodu yazmayan bir ya da birden fazla kişi tarafından üretime alınmadan önce okunması işlemidir. Yaygın modern biçimi, sürüm kontrol sisteminde açılan bir değişiklik önerisinin (*pull request*, kısaca PR) bir onaylayıcı tarafından incelenip birleştirmeye izin verilmesi biçimindedir.

Alanda yerleşik iki kabul vardır. Birincisi, kod incelemesinin yazılım kalitesini artırdığıdır. İkincisi, incelemenin kaldırılmasının kalitede bozulmayla sonuçlanacağıdır. Her iki kabul de, incelemenin tek bir işlevi olduğu ve bu işlevin ölçüldüğü varsayımına dayanır. Aşağıda kurulan çerçeve bu varsayımdan ayrılır: kod incelemesi tek bir işlev değil, tarihsel olarak farklı zamanlarda eklenmiş dört ayrı işlevin aynı ritüel altında birleşmiş hâlidir. Bu işlevlerin ayrıştırılması, hangisinin devredilebilir olduğunu ve hangisinin devredilemediğini görünür kılar.

Merkezî terimler şöyle tanımlanır.

Kusur (*defect*), programın belirtilen davranıştan sapmasına yol açan kod özelliğidir. Stil tercihi, isimlendirme ya da mimari beğeni kusur değildir.

Geri alınabilirlik (*reversibility*), bir değişiklik hatalı çıktığında sürümün geri alınmasıyla sistemin önceki durumuna dönüp dönmediğidir. Kod her zaman geri alınabilir; kodun ürettiği yan etki her zaman geri alınamaz.

Etki alanı (*blast radius*), hatalı bir değişikliğin kaç kullanıcıyı ya da sistemin kaç bileşenini etkilediğidir.

Örtük bilgi (*tacit knowledge*), bir sistemin neden mevcut biçimde olduğuna dair, hiçbir yerde yazılı olmayan ve yalnızca ilgili kararı almış kişide bulunan bilgidir. Kodun *ne yaptığı* kodun kendisinde bulunur; *neden öyle olduğu* bulunmaz.

Birleştirme sonrası inceleme (*post-merge review*), kodun ana dala girmesinin bir onaya bağlanmadığı, incelemenin varsa birleştirmeden sonra yapıldığı modeldir. Bu modelde inceleme yorumları bir kapı değil, öneri statüsündedir.

Çerçeve, sürekli teslimat yapabilen, sürümü kısa sürede geri alabilen ve üretim ortamını gözlemleyebilen yazılım organizasyonlarını kapsar. Gömülü sistemler, tıbbi cihaz yazılımı, havacılık ve benzeri düzenlemeye tabi alanlar kapsam dışındadır; bu alanlarda sürümün geri alınması ya imkânsızdır ya da bağımsız bir onay zorunluluğu yasal olarak dayatılmıştır.

Kod incelemesinin kökeni

Fagan incelemesi

Formal kod incelemesini yöntem olarak tanımlayan çalışma Michael Fagan tarafından IBM’de yapılmış ve 1976’da yayımlanmıştır (Fagan, 1976). Yöntemin adı *inspection*’dır ve bugünkü PR incelemesiyle biçimsel olarak çok az ortak yanı vardır.

Yöntemin dayandığı ekonomik gerekçe dönemin koşullarındadır. Yazılım manyetik bant ya da disket üzerinde dağıtılıyor, müşteri sahasında kuruluyordu. Üretime çıkmış bir hatanın düzeltilmesi yeni bir dağıtım döngüsü gerektiriyordu ve sürümü geri alma diye bir işlem mevcut değildi. Bu koşullarda hatayı kod üretime çıkmadan yakalamaya adam-saat harcamak rasyoneldi.

Rollerin ayrımı

İncelemenin ayırt edici özelliği, katılımcı rollerinin kesin biçimde ayrılmış olmasıdır.

- **Yazar** (*author*) kodu yazan kişidir. Toplantıda kendiliğinden konuşmaz; yalnızca kendisine soru yöneltildiğinde cevap verir.
- **Okuyucu** (*reader*) yazardan farklı bir kişidir. Kodu kendi cümleleriyle, satır satır, sesli olarak anlatır.
- **İnceleyici** (*inspector*) dinler ve kusur arar.
- **Moderatör** (*moderator*) süreci yönetir, kusurları kaydeder ve kurallardan sapmayı keser.

Okuyucunun yazardan farklı olması yöntemin merkezindedir. Kodu yazan kişi kodu anlattığında kendi niyetini anlatır; kodda fiilen yazan şeyi değil. Başka bir kişi okuduğunda niyet ile metin arasındaki fark ortaya çıkar. Kusur çoğunlukla tam olarak bu farkın kendisidir.

Süreç kuralları

Yöntemin üretkenliğini sağlayan, toplantının kendisinden çok onu çevreleyen kurallardır.

- **Bireysel hazırlık zorunludur.** Katılımcılar toplantıdan önce kodu tek başlarına okur. Hazırlanmamış katılımcı varsa toplantı ertelenir.
- **Çözüm tartışılmaz.** Toplantının tek işlevi kusuru *bulmaktır*. “Şöyle yazılsa daha iyi olurdu” biçimindeki katkılar yasaktır ve moderatör tarafından kesilir. Düzeltmenin nasıl yapılacağı, toplantıdan sonra tek başına yazara aittir.

- **Hız sınırlanmıştır.** İnceleme hızı saat başına yaklaşık yüz ilâ iki yüz satırla sınırlıdır. Bu sınırın üstünde kusur bulma oranının düştüğü ölçülmüştür. Katılımcılar yavaş okumaya yapısal olarak zorlanır.
- **Kontrol listesi kullanılır.** Serbest okuma yerine sabit bir liste üzerinden bakılır: sınır koşulları, kaynak serbest bırakma, sayısal taşma, indeks sınırı gibi maddeler.
- **Liste kendini besler.** Üretimde yakalanan her yeni kusur tipi kontrol listesine kalıcı madde olarak eklenir. Süreç böylece zamanla kuruma özgü hâle gelir.
- **Sonuçlar ölçülür.** Kaç kusur bulunduğu, hangi aşamada bulunduğu ve saat başına düşen kusur sayısı kayıt altına alınır.

Somut bir örnek

Yöntemin işleyişi, bir faiz hesaplama modülünün incelenmesi üzerinden görülebilir. Okuyucu, döngünün her ay için bakiyeyi faiz oranıyla çarptığını ve on iki kez döndüğünü anlatır. İnceleyici, döngü koşulunun sınır değerini dışarıda bıraktığını ve fiilen on bir ayın hesaplandığını belirtir. Moderatör kusuru kaydeder. Yazar açıklama yapmak ister; moderatör sözü keser ve okuyucu devam eder.

Bu üç saniyelik alışverişte olan şey dikkate değerdir. Kusur kaydedilmiş, tartışma açılmamış, yazar savunmaya geçmemiş, kimse alternatif bir yazım önermemiştir. Toplantı sonunda moderatör kusur listesini yazara verir; yazar düzeltmeyi tek başına yapar ve moderatör yalnızca maddelerin kapanıp kapanmadığına bakar. Yeniden toplantı yapılmaz.

Modern kod incelemesi ve işlev kayması

Gerekçenin düşmesi

Sürekli tümleştirme, otomatik test, özellik bayrağı (*feature flag*) ve tek işlemle sürüm geri alma yaygınlaştıkça, incelemenin dayandığı ekonomik denklem geçersizleşmiştir. Hatanın üretime çıkması artık dakikalar içinde geri alınabilen bir olaydır. Buna karşılık pratik ortadan kalkmamış, biçim değiştirerek sürmüştür.

Fagan kurallarının kaybı

Bugün PR incelemesi olarak uygulanan şey, Fagan yönteminin hafifletilmiş bir sürümü değildir; yöntemin üretkenliğini sağlayan kuralların tamamına yakını kaybetmiş, farklı bir pratiktir.

Fagan incelemesi	Modern PR incelemesi
Okuyucu yazardan farklı	Okuyan ve değerlendiren aynı kişi
Yazar savunma yapmaz	Yazar her yoruma cevap verir
Çözüm önerme yasak	Yorumların çoğu çözüm önerisi
Saatte 100–200 satır	Sınır yok, yoğunlukla saatte binlerce satır
Bireysel hazırlık zorunlu	Hazırlık kavramı yok
Kontrol listesi	Serbest bakış
Moderatör tartışmayı keser	Moderatör yok
Onay/veto yetkisi yok	Onay birleştirmenin ön koşulu

Son satır özellikle önemlidir. Fagan yönteminde inceleyicinin kodu durdurma yetkisi bulunmaz; yalnızca kusur kaydeder. Modern biçimde onaylayıcı, birleştirmeyi engelleyebilir. Bu, yöntemde bulunmayan ve sonradan eklenmiş bir yetkidir.

Ampirik bulgular

Bacchelli ve Bird, Microsoft'ta yürüttükleri çalışmada geliştiricilerin kod incelemesinden ne beklediğini sormuş, ardından incelemenin fiilen ne ürettiğini ölçmüştür (Bacchelli ve Bird, 2013). Beklenti kusur bulmaktır. Ölçülen çıktı farklıdır: inceleme yorumlarının büyük çoğunluğu kusur değil, bakım kolaylığı ve alternatif çözüm önerileriyle ilgilidir. Çalışmanın öne çıkardığı asıl bulgu, incelemenin en değerli çıktısının kimsenin beklemediği bir yan etki olmasıdır: bilgi transferi ve kod üzerinde ortak farkındalık.

Sadowski ve arkadaşlarının Google'daki uygulamayı incelediği çalışma bu bulguyu tamamlar (Sadowski ve ark., 2018). Google'da incelemenin işlerliğini sağlayan şey titizlik değil hafifliktir; değişikliklerin yoğunluğu küçüktür, genellikle tek inceleyici atanır ve hafta içi değişikliklerin yaklaşık %70'i incelemeye gönderildikten sonraki yirmi dört saat içinde birleştirilir. Çalışmada inceleme, bir kalite kapısından çok eğitim ve ortak sahiplenme mekanizması olarak konumlandırılmıştır.

Sürekli teslimat literatüründe, harici onay kurulları ve ağır değişiklik onay süreçlerinin teslimat performansı ile negatif ilişkili olduğu, buna karşılık hafif akran incelemesinin olumsuz etki üretmediği raporlanmıştır (Forsgren, Humble ve Kim, 2018).

Bu bulguların ortak işareti şudur: incelemenin faydasını belirleyen değişken, incelemenin ne kadar titiz yapıldığı değil, geri bildirimin ne kadar hızlı geldiği ve değişikliğin ne kadar küçük olduğudur.

İncelemenin dört işlevi

Kod incelemesinin tek bir amaç etrafında tartışılması, farklı işlevlerinin aynı ada bağlanmış olmasından kaynaklanır. Ayrıştırıldığında dört işlev görünür.

Kusur yakalama

Yöntemin resmî ve tarihsel amacıdır. Bu işlev bugün büyük ölçüde otomasyona devredilmiştir: birim testi, tip denetleyicisi (*type checker*), statik çözümleme ve bağımlılık taraması, insan okumasının yakalayabileceği kusur sınıflarının çoğunu daha güvenilir ve daha ucuz biçimde yakalar. Otomasyonun üstünlüğü tekrarlanabilirliktedir: insan okuması yorgunluğa ve dikkate bağlıdır, otomatik denetim her çalıştırmada aynı sonucu verir.

Biçimsel tutarlılık

İsimlendirme, biçimlendirme ve stil tutarlılığıdır. Bu işlev bütünüyle otomasyona devredilebilir ve devredilmesi gerekir. Bir ekte stil tartışması çıkıyorsa bunun nedeni kuralın olmaması değil, kuralın *yazılı* olmamasıdır. Biçimlendirici ve dizim denetleyicisi (*linter*) kuralı yazılı ve tartışılmaz hâle getirdiği anda bu tartışma kaynağı ortadan kalkar.

Bilgi dağılımı

En değerli ve en az planlanmış işlevdir. Kodu okuyan kişi o alanı bilmiyorsa, okuduktan sonra kısmen bilir hâle gelir. Bu birikim zamanla kod tabanının zihinsel haritasını oluşturur.

Ancak bu mekanizmanın işlemesi dört koşula bağlıdır ve pratikte bu dördü nadiren birlikte sağlanır.

- İnceleyici o alanı *bilmiyor* olmalıdır. Uygulamada tam tersi yapılır: en hızlı bakacağı için dosyanın sahibine atanır. Bilen okur, bilmeyen okumaz; bilgi dağılmaz, mevcut yoğunlaşma pekişir.
- Kod gerçekten okunmalıdır. “LGTM” (*looks good to me*) biçimindeki içeriksiz onayın yaygınlığı, bunun sıklıkla gerçekleşmediğinin göstergesidir.
- Değişiklik küçük olmalıdır. Sekiz yüz satırlık bir öneri okunmaz, taranır; tarama zihinsel harita bırakmaz.
- Okuma yakın zamanlı olmalıdır. Bir kez okunup aylarca dokunulmayan kod hatırlanmaz.

Ayrıca aktarılan bilginin kapsamı sınırlıdır. Okumakla öğrenilen şey kodun *ne yaptığıdır*. Kodun *neden öyle olduğu* kodda yazmadığı için aktarılmaz. Dolayısıyla kod incelemesi, örtük bilgi sorununun kısmi ve zayıf bir çözümüdür; tam çözümü hiçbir zaman olmamıştır.

Onay ve veto

Dördüncü işlev, incelemenin organizasyonda aldığı biçimdir ve ilk üçünden bağımsızdır. Onaylayıcının birleştirmeyi durdurabilmesi, teknik bir mekanizma değil hiyerarşik bir yetkidir.

Bu yetkinin varlığı, inceleme yorumlarının anlamını değiştirir. Veto yetkisi olan birinin ağzından çıkan “şöyle yapsak nasıl olur” cümlesi biçimsel olarak sorudur, işlevsel olarak talimattır. Reddetme maliyeti asimetriktir: kabul eden tartışmayı bitirir, reddeden gerekçe üretmek zorunda kalır. Alan gözlemlerinde “ego çatışması” olarak adlandırılan olgunun kaynağı bu asimetridir.

Fagan yönteminde bu yetkinin bulunmaması ve çözüm önermenin yasak olması tesadüf değildir; ikisi birlikte tartışmanın çıkacağı zemini ortadan kaldırır.

Değişiklik sınıflandırması

İnceleme yükümlülüğünün nerede gerekli olduğu, değişikliğin önem derecesine göre değil, iki ölçülebilir eksene göre belirlenir.

Eksenler

Birinci eksen **geri alınabilirlik**dir. Sorulan soru şudur: değişiklik hatalı çıkarsa sürümü geri almak yeterli midir? Kod her zaman geri alınır; kodun ürettiği kalıcı yan etki geri alınmaz.

İkinci eksen **etki alanı**dır: hata kaç kullanıcıyı ya da kaç bileşeni etkiler.

Buradaki ayrım, sezgisel “önemli iş” kavramından farklıdır ve fark kritiktir. Kimlik doğrulama önemli bir alandır ancak geri alınabilir: hatalı bir yetki değişikliği sürüm geri alınıp oturum belirteçleri iptal edilerek düzeltilir. Buna karşılık görece önemsiz görünen bir veritabanı göçü (*migration*) geri alınamaz. Sınıfı belirleyen önem değil, dönüşün mümkün olup olmadığıdır.

Dört sınıf

Sınıf	Konum	Güvence yöntemi
Sınıf 1	Geri alınabilir, dar etki	Yok. Gözleme yeterli
Sınıf 2	Geri alınabilir, geniş etki	Özellik bayrağı; kapalı gelir
Sınıf 3	Geri alınamaz, dar etki	Otomatik test zorunlu
Sınıf 4	Geri alınamaz, geniş etki	Tasarımla geri alınabilir kılma

Sınıf 1 arayüz değişikliklerini, günlük kaydını ve yeniden düzenlemeyi (*refactoring*) kapsar. Sınıf 2 yeni akışları ve dış servis tümleştirmelerini kapsar. Sınıf 3 kimlik doğrulama ve yetkilendirme gibi geri dönüşü zahmetli ama mümkün olan alanları kapsar. Sınıf 4 veritabanı göçlerini, ödeme işlemlerini ve toplu veri silmeyi kapsar.

Sınıf 4’ün ayrışıklığı

Sınıf 1–3’te geçerli olan varsayım şudur: hata çıkarsa fark edilir ve geri alınır. Sınıf 4’te bu varsayım geçersizdir.

Bir kolonu düşüren göç çalıştığında, sürümün geri alınması kodu eski hâline döndürür ancak kolondaki veri geri gelmez. Yeniden deneme mantığındaki bir hata aynı ödemeyi iki kez gönderdiğinde, kodun düzeltilmesi çekilmiş parayı iade etmez. Toplu silme işlemi yanlış filtreyle çalıştığında, kodun geri alınması kayıtları geri getirmez.

Bu sınıfta test de yeterli değildir; test yalnızca önceden düşünülmüş senaryoyu yakalar. Doğru müdahale, hatayı yakalamak değil hatanın kalıcı olmasını yapısal olarak imkânsız kılmaktır.

- **Göç:** Kolon silme işlemi, ekleme işlemiyle aynı sürümde yapılmaz. Önce yeni kolon eklenir, kod bir süre her iki kolonu da yazar, eski kolon ayrı bir sürümde düşürülür. Her adım tek başına geri alınabilir hâle gelir.
- **Ödeme:** Her ödeme isteğine benzersiz bir eşsizlik anahtarı (*idempotency key*) eklenir. Aynı anahtar ikinci kez geldiğinde sağlayıcı işlemi tekrarlamaz, ilk sonucu döndürür. Yeniden deneme hatası artık çift çekim üretmez.
- **Silme:** Fiziksel silme yerine işaretleme kullanılır. Yanlıklık hâlinde işaret kaldırılır.

Bu müdahalelerin ortak özelliği, bir defalık olmalarıdır. Bir alan yapısal olarak güvenceye alındığında o alan Sınıf 4'ten Sınıf 1'e düşer ve sonrasında normal hızda geliştirilir. Sınıflandırmanın amacı sürekli bir denetim yükü kurmak değil, sınıfı değiştirmektir.

Hata sınıflandırması

Aynı eksen hatalar için de geçerlidir. Gürültülü hata, uyarı üreten ve derhal fark edilen hatadır; Sınıf 1 davranışı gösterir. Sessiz hata, yanlış bir değeri kaydeden ve hiçbir uyarı üretmeyen hatadır; fark edildiğinde etkisi çoktan yayılmıştır ve bu nedenle fiilen geri alınamaz. Sessizlik, bir hatayı geri alınamaz kılan asıl özelliktir.

İnceleme sonrası bir modelin kurulması

İnceleme yükümlülüğünün kaldırılması, incelemenin taşıdığı işlevlerin kaldırılması anlamına gelmez. İşlevler devredilir; devredilmediğinde model güvenlik ağı olmadan işler.

İnsan kapısının otomatik kapıyla değiştirilmesi

- Sürekli tümleştirme zorunlu kılınır: test, biçim denetimi ve tip denetimi geçmeden birleştirme yapılmaz. Kapı vardır ancak insan değildir.
- Değişiklikler küçük tutulur. Birleştirme sonrası modelin tek çalışma koşulu budur; büyük değişiklik geri alınmaz.
- Geri alma bir suçlama değil rutin bir işlem olarak konumlandırılır. Sorun çıktığında tartışma açılmaz, sürüm geri alınır.
- Riskli değişiklikler özellik bayrağı arkasında, kapalı durumda birleştirilir. Açma kararı ayrı bir işlemdir.

Gözlemlenebilirlik

Kodun okunmadığı bir modelde tek güvenlik ağı, çalışan sistemin durumunu bildirmesidir. Hata izleme, eşik aşımında uyarı üretme ve tek işlemle sürüm geri alma bu modelin taşıyıcı bileşenleridir. Bunlar mevcutsa kodu okumama kararı savunulabilir bir mühendislik tercihidir; mevcut değilse aynı karar ihmaldir.

Sınırlı test kapsamı

Kapsamlı test yazımı yerine üç yola odaklanılır: ödeme akışı, kimlik doğrulama ve yetkilendirme, veri değiştiren ya da silen işlemler. Bu üç alan sessizce bozulduğu için gözlemlemeyle yakalanamaz. Kalan alanlarda test eksikliği gözlemlemeyle telafi edilebilir.

Veto yetkisinin kalkmasının etkisi

Birleştirme sonrası modelde kod ana dala girmiş durumdadır. Bu nedenle inceleme yorumu artık talimat olamaz; öneri statüsündedir. Yorum yapan kişi değişmemiş, yorumun içeriği değişmemiş, yalnızca yorumun arkasındaki yetki kalkmıştır. Alan gözlemlerinde çatışma kaynağı olarak görülen olgu, tam olarak bu yetkinin kalkmasıyla ortadan kalkar.

Örtük bilginin taşınması

Sorunun doğru tanımı

Kodun okunmadığı bir modelde ödenen bedel kod kalitesinde değil, bilgi dağılımında ödenir ve bu bedel gecikmelidir. Ekip yetkin olduğu sürece hiçbir sinyal üretmez; sinyal, bir sistemin belirli bir parçasını yalnızca bir kişinin anladığı ortaya çıktığında, yani o kişi ayrıldığında ya da erişilemez olduğunda ani biçimde gelir.

Bu risk ekip büyüklüğüyle doğrusal değil, kare olarak artar. Brooks'un işaret ettiği ilişki gereği, bir ekipteki iletişim kanalı sayısı $n(n-1)/2$ ile büyür (Brooks, 1975). Dört kişilik bir ekipte altı kanal bulunurken sekiz kişilik ekipte yirmi sekiz kanal bulunur. Ekip iki katına çıkarken "kim kimin ne yaptığını biliyor" ilişkisi dört katına çıkar. İnsan bunu takip edemediği için ekip kendiliğinden alt kümeler ayrılır: iki üç kişilik adalar oluşur, her ada kendi alanını bilir. Bu ayrışma sessizdir; kimse bilmediğini beyan etmez, yalnızca o dosyaya dokunmaz.

Büyük dil modelinin kapsadığı ve kapsamadığı alan

Kod tabanının bir büyük dil modeliyle taranıp açıklattırılması, kodun *ne yaptığı* boyutunu büyük ölçüde kurtarılabilir kılmıştır. Daha önce adam-ay süren tanışma süreci saatler mertebesine inmiştir. Bu değişim, bilgi dağılımı riskinin bir bölümünü gerçekten ortadan kaldırır.

Kapsanmayan boyut, kodun *neden öyle olduğu*dur. Bir yeniden deneme sayısının neden üç olduğu, bir tabloda neden alışılmadık bir kolon bulunduğu, yavaş çalışan bir uç noktanın neden düzeltilmediği kodda yazmaz. Model bu bilgiyi üretmez; hiçbir yerde yazılı olmadığı için çıkarım yapacağı bir kaynak yoktur.

Ayrılan kişiyle giden şey kod bilgisi değil, bu gerekçe bilgisidir. Pratik sonucu tipik olarak şudur: yeni gelen kişi alışılmadık gördüğü yapıyı temizler ve daha önce çözülmüş bir sorun geri döner. Sorunun neden geri döndüğünü kimse bilmez, çünkü ilk çözümü yapan kişi ayrılmıştır.

Arşivin yetersizliği ve işaretçi kavramı

Kurumsal iletişimin kayıt altına alınması, yani toplantı özetleri ve yazışma arşivleri, bu sorunu tek başına çözmez. Sorun bilginin kayıtlı olmaması değil, arayan kişinin *arayacağını bilmemesidir*. İlgili satırı alışılmadık görmediği için soru sormaz; soru sormadığı için arama yapmaz.

Yapısal neden şudur: arşiv zamana göre dizilmiştir, kod ise konuma göre. İkisi arasında bir bağ yoktur.

Bu bağ kuran yapıya **işaretçi** (*pointer*) denebilir: kodun içine bırakılan, o noktanın bir gerekçesi olduğunu ve gerekçenin nerede bulunacağını bildiren tek satırlık kayıt.

```
// 3 deneme - musteri API'si 4'te 429 donuyor (DECISIONS.md #12)
maxRetries := 3
```

İşaretçi iki iş yapar. Okuyana o noktanın tesadüf olmadığını bildirir ve gerekçenin konumunu verir. İşaretçisiz hâlde okuyan yalnızca bir sabit görür ve durmaz.

Uygulama kuralı basittir: *olağan kod açıklamasız bırakılır, olağandışı kod işaretçili yazılır*. Olağandışı olan, bir başkasının silmeye ya da sadeleştirmeye kalkışabileceği her yapıdır.

İşaretçilerin işaret ettiği hedef, kararı ve gerekçesini tarihiyle birlikte tutan bir karar kaydıdır. Üç satır yeterlidir: alınan karar, alınma sebebi, ortaya çıkan sonuç.

Bu yapı aynı zamanda büyük dil modelinin işlerliğini belirler. Model, işaretçiyi gördüğünde karar kaydını da okuyabilir ve gerekçeyi aktarabilir. İşaretçi yoksa model de alışılmadık olanı alışılmadık göremez ve arşivi aramayı bilemez. Dolayısıyla işaretçi, bilginin modele erişilebilir hâle gelmesini sağlayan bileşendir; süs değildir.

Yapılandırılmış otomatik inceleme

Fagan yönteminin kusur bulma işlevi, rol ayrımı korunmak kaydıyla bir büyük dil modeline devredilebilir. Kritik nokta, rollerin *ayrı çağrılarda* kurulmasıdır; tek bir istem içinde birleştirildiğinde model kendi anlatımını doğrular ve okuyucu ile inceleyici arasındaki fark ortadan kalkar.

Okuyucu çağrısı

Modelden kodu satır satır kendi ifadeleriyle anlatması istenir. Hata araması, öneri yapması ve yorumda bulunması açıkça yasaklanır. Çıktı, kodun ne yapması gerektiğinin değil, fiilen ne yaptığının dökümüdür.

İnceleyici çağrısı

Ayrı bir bağlamda başlatılır. Modele kod, okuyucu çağrısının çıktısı ve kontrol listesi birlikte verilir. Kurallar açıkça belirtilir: çözüm önerilmez, stil ve isimlendirme yorumu yapılmaz, her bulgu satır numarası, tip ve önem derecesiyle tek cümlede yazılır, emin olunmayan bulgu yazılmaz.

Kontrol listesi Fagan'daki işleviyle aynıdır: sınır koşulları, boş ve eksik veri, sayısal taşma, indeks sınırı, hata yolunun ele alınıp alınmadığı, geri alınamaz işlem varlığı, eşzamanlılık.

Moderatör ve yazar işlevleri

Bu iki rol insanda kalır. Çözüm önerme yasağını çiğneyen bulgular elenir, önemsiz bulgular biriktirilir, kalan bulgular düzeltilir. Düzeltmenin nasıl yapılacağı, Fagan yöntemindeki gibi, tümüyle yazara aittir.

Listenin beslenmesi

Yöntemin en değerli parçası, kontrol listesinin üretimdeki hatalarla beslenmesidir. Üretimde ortaya çıkan her yeni hata tipi listeye kalıcı madde olarak eklenir. Liste zamanla kod tabanına özgü hâle gelir ve genel bir modelin bilemeyeceği bilgiyi taşır.

Kapsam

Bu düzenek her değişiklikte çalıştırılmaz; aksi hâlde kaldırılan yükümlülüğün yerine yenisi kurulmuş olur. Sınıf 3 ve Sınıf 4 değişikliklerde, yani zaten durulması gereken yerlerde uygulanır.

Sınırlılık açıktır: düzenek Fagan yönteminin kusur bulma işlevini taklit eder, bilgi dağılımı işlevini etmez. Okuyucu çağrısının çıktısı bu boşluğu kısmen kapatabilir; kodun yanına yazılı açıklama olarak bırakıldığında kalıcı bir kayda dönüşür.

Sonuç

Kod incelemesinin tartışılmasını zorlaştıran şey, tek bir ada bağlanmış dört ayrı işlevin ayrıştırılmamış olmasıdır. Ayrıştırıldığında tartışma teknik olarak çözülebilir hâle gelir: kusur yakalama ve biçimsel tutarlılık otomasyona devredilir, onay ve veto işlevi zaten fayda üretmediği için kaldırılır, geriye yalnızca bilgi dağılımı kalır. İnceleme yükümlülüğünün kaldırılıp kaldırılamayacağı sorusu, bütünüyle bu dördüncü işlevin başka bir yolla karşılanıp karşılanmadığına indirgenir.

Bu indirgeme, sorunun eksenini de değiştirir. “İnceleme var mı yok mu” ikilisi karar için yararlı bir eksen değildir; yararlı eksen, geri bildirimin gecikme süresi ve değişikliğin büyüklüğüdür. Google örneğinde incelemeyi işler kılan şeyin titizlik değil hafiflik olması bunun göstergesidir.

İncelemenin kaldırılabilceği koşullar sıralanabilir: sürekli tümleştirmenin zorunlu kılınması, hata izleme ve uyarı düzeneğinin kurulması, hızlı geri alma yeteneği, üç kritik yolda test kapsamı, geri alınamaz alanların tasarımla güvenceye alınması ve olağandışı kodun işaretçilerle kayıt altına alınması. Bu altı koşuldan biri eksikse ortaya çıkan durum “incelemeden çıkmış bir organizasyon” değil, güvenlik ağı olmadan işleyen bir organizasyondur.

Sınırlılıklar

Çerçevenin geçerliliği birkaç noktada bulanıklaşır.

Birincisi, gözlemsel dayanağın zayıflığıdır. “İnceleme yapılan şirketlerde proje başarısız oldu, yapılmayanda olmadı” biçimindeki gözlem, örneklem küçüklüğü ve karıştırıcı değişkenler nedeniyle nedensellik kurmaya elverişli değildir. Ağır süreç çoğunlukla bir neden değil bir belirtidir: kurumlar zorunlu inceleme, çoklu onaylayıcı ve kontrol listesi gibi mekanizmaları güven kaybı yaşandıktan sonra koyar. Bu durumda sıkı inceleme ile başarısız proje, aynı altta yatan işlev bozukluğunun iki ayrı çıktısı olabilir. Aynı mantık ters yönde de geçerlidir: incelemesiz bir ekibin sorunsuz işlemesi, incelemenin gereksizliğini değil, ekibin küçük ve düşük devirli olduğunu gösterir.

İkincisi, işaretçi disiplininin kırılabilirliğidir. Model, işaretçilerin fiilen yazılması varsayımına dayanır. Bu disiplin uygulanmadığında diğer beş bileşen örtük bilgi kaybını karşılamaz ve incelemesizlik gerçek anlamda çıplak kalır.

Üçüncüsü, kapsam sınırıdır. Çerçeve, sürümün hızla geri alınabildiği ortamlar için kurulmuştur. Düzenlemeye tabi alanlarda bağımsız onay hukuki bir yükümlülüktür ve teknik gerekçeyle kaldırılamaz.

Açık kalan sorular

İnceleme yükümlülüğünün kaldırıldığı bir organizasyonda bilgi dağılımının fiilen sürüp sürmediği doğrudan ölçülebilir mi? Kod sahipliğinin yoğunlaşması, kimin hangi dosyaya dokunduğu üzerinden izlenebilir bir metrik olarak kurulabilir görünmektedir, ancak bu metriğin örtük bilgi kaybını ne ölçüde temsil ettiği açık değildir.

İkinci soru, büyük dil modelinin bilgi dağılımı işlevini ne ölçüde devraldığıdır. Modelin kodu açıklama yeteneği, sorulacak sorunun bilinmesini gerektirir. Kod tabanına aşina olmayan bir kişinin, işaretçilerin bulunduğu bir kod tabanında modeli ne kadar etkin kullanabildiği ampirik olarak ölçülmemiştir.

Üçüncü soru, sınıflandırmanın ölçeklenebilirliğidir. Geri alınabilirlik ekseninin küçük ekiplerde sezgisel biçimde uygulanabildiği görülmektedir; ekip büyüdüğünde bu değerlendirmenin tutarlı biçimde yapıp yapılamayacağı, yani sınıflandırmanın kendisinin bir süreç dönüşüp dönüşmeyeceği belirsizdir.

Kaynaklar

- Bacchelli, A. ve Bird, C. (2013). Expectations, Outcomes, and Challenges of Modern Code Review. *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, 712–721.
- Brooks, F. P. (1975). *The Mythical Man-Month: Essays on Software Engineering*. Addison-Wesley.
- Fagan, M. E. (1976). Design and Code Inspections to Reduce Errors in Program Development. *IBM Systems Journal*, 15(3), 182–211.
- Forsgren, N., Humble, J. ve Kim, G. (2018). *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press.
- Sadowski, C., Söderberg, E., Church, L., Sipko, M. ve Bacchelli, A. (2018). Modern Code Review: A Case Study at Google. *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 181–190.